



**Politecnico
di Torino**

*Master's degree course in
Digital Skills for Sustainable Societal Transitions
(LM-91)*

*Applied AI and Machine Learning
(01VIATD)*

Report title:

Exploring LLMs for News Classification

Professor:

Gianvito URGESE

Assistant Professors:

Vittorio FRA

Walter GALLEG0

Student:

Davide OLIVETTI

Student's code:

s348090

Accademic year 2024/2025

1 Introduction

This laboratory explores the interplay between Large Language Models (LLMs) and the KNIME analytics platform for addressing a real-world classification problem. The task involves leveraging the capabilities of LLMs to support the construction of KNIME workflows aimed at categorizing news articles based on their publishers. This scenario tests the implementation of classification models such as Multi-Layer Perceptrons (MLPs) and Decision Trees, while highlighting the necessity of a critical approach when relying on LLMs for workflow design and data preprocessing.

The dataset at the center of this laboratory, `newsCorpora.csv`, was sourced from a news aggregator and contains a rich collection of web page references spanning from March 10 to August 10 of 2014. Due to restrictions on full content usage, the dataset includes only metadata, offering an opportunity to classify news items based solely on the publisher feature. This constraint underscores the importance of creative feature engineering and thoughtful preprocessing strategies.

A key consideration in this dataset is the imbalance across the four news categories: business, science and technology, entertainment, and health. Such imbalance can significantly affect the performance of classification models and requires careful handling during both model training and evaluation. The distribution of articles among the categories is summarized in the following table:

Category	Label	Number of News Articles
Business	b	152.469
Science and Technology	t	108.344
Entertainment	e	115.967
Health	m	45.639

Table 1: Distribution of news articles across categories in the `newsCorpora.csv` dataset.

To prepare the dataset for the classification models, the publisher information is converted from string-based identifiers into numeric representations, ensuring each unique publisher is mapped to a distinct numeric value. Subsequent preprocessing steps, including normalization (where applicable) and data partitioning, establish a robust foundation for model training and testing.

Throughout the workflow design and implementation, the reliability of LLM-generated insights must be approached with caution. While LLMs can provide helpful suggestions, their tendency to hallucinate or produce imprecise outputs underscores the need for careful verification and critical analysis of their contributions.

This laboratory highlights the application of AI and machine learning techniques within a real-world analytics framework, balancing the power of modern AI tools with the rigor of classical algorithmic models. Ultimately, it emphasizes not only the technical challenges of data-driven classification but also the need for careful attention to data quality, category imbalance, and the interpretability of AI-assisted processes.

2 Multi-Layer Perceptron

2.1 Rprop MLP Learner

The Multi-Layer Perceptron (MLP) model was chosen as the classification algorithm for this task, leveraging the numeric representation of the `PUBLISHER` feature to predict the news `CATEGORY`. In order to prepare the data appropriately, several preprocessing steps were carried out, including conversion of publisher names to numeric identifiers, handling missing values, and normalization. The MLP algorithm requires numeric input data bounded between 0 and 1, which was achieved using a normalization step.

The dataset was partitioned using stratified sampling, ensuring that the relative proportions of each class (`b`, `t`, `e`, `m`) remained consistent between the training and test sets. A random seed (42) was utilized throughout the pipeline to maintain reproducibility.

The MLP model’s configuration included:

- **Maximum number of iterations:** 200. This parameter specifies how many times the algorithm’s internal update rules (weight updates based on the backpropagation algorithm) are repeated. A moderate value like 200 balances the need for convergence with computational cost.
- **Hidden layers:** a parameter optimization loop was employed to explore between 1 and 3 hidden layers. Each hidden layer adds a level of abstraction, potentially capturing more complex patterns in the data.
- **Neurons per layer:** similarly, the number of neurons per hidden layer was varied between 48 and 56 in steps of 4. This range was chosen to explore a sufficiently rich network structure without incurring excessive overfitting or computational expense.

The parameter optimization loop identified the best-performing configuration as 3 hidden layers with 56 neurons per layer. However, the resulting classification accuracy remained modest, achieving approximately 47% with a Cohen’s kappa of 0,229. The performance details, presented in Table 2, highlight the confusion matrix of the classification results.

Table 2: Confusion matrix of the MLP classifier on the test set

	b	t	e	m
b	15.433	6.411	12.946	0
t	6.629	13.382	12.492	0
e	5.709	9.223	30.809	0
m	2.783	3.494	7.415	0

The confusion matrix reveals that the classifier never predicts the `m` category (`health`), with no instances assigned to this class in the output. This absence can be attributed to two main factors:

1. **Imbalanced class distribution:** as shown in the dataset distribution, the `m` category is significantly underrepresented compared to the other classes. The MLP tends to bias predictions towards more represented classes, resulting in poor generalization for the underrepresented class.
2. **Feature limitation:** only the `PUBLISHER` feature was used for classification. This feature alone may not contain sufficient discriminatory information to accurately separate the `m` category from the others, especially given the smaller number of samples available for that class.

Despite adjusting the network depth and width via the parameter optimization loop, the classifier’s performance plateaued around the 47% mark. This outcome underscores the limitations of using a single feature (PUBLISHER) and suggests that additional features such as TITLE or STORY might be necessary to achieve more robust classification, particularly for underrepresented classes.

Overall, while the MLP was able to capture certain patterns in the data, its performance was constrained by class imbalance and limited feature informativeness.

2.2 Keras Network Learner

In order to improve the classification an alternative approach based on the *Keras Network Learner* was adopted. It relies on the Keras deep learning library, offering additional flexibility and parameter tuning capabilities.

This learner was integrated into a refined workflow pipeline that include a *One to Many* node to handle the categorical output, transforming the single target column into four distinct binary columns, each representing one of the classes: *b*, *t*, *e*, and *m*. This preprocessing step was crucial to adapt the data to the format required by the Keras learner.

The *Keras Network Learner* was configured with 3 hidden layers and 56 neurons per layer, aligning with the architecture explored in the previous *RProp MLP Learner*. However, the *Keras* learner introduced two additional hyperparameters:

- **Epochs** parameter, set to 30, defines the number of times the entire training dataset passes through the network during training, providing multiple opportunities for the model to learn from the data.
- **Learning Rate**, set to 0,001, controls the magnitude of the adjustments made to the model’s weights during each update step, influencing the convergence speed and stability of the training process.

A summary of the differences between the *Keras Network Learner* and the *RProp MLP Learner* can be presented as follows:

Table 3: Comparison of *RProp MLP Learner* and *Keras Network Learner* settings

Parameter	RProp MLP Learner	Keras Network Learner
Hidden layers	3	3
Neurons per layer	56	56
Iterations/Epochs	200 iterations	30 epochs
Learning rate	–	0,001
Category output handling	Direct mapping	One to Many

When evaluating the results obtained through the *Keras Network Learner*, it emerged that the classification performance exhibited a slight improvement over the previous learner. The accuracy reached 49,159%, surpassing the previous 47,050% obtained with the *RProp MLP Learner*. The Cohen’s kappa coefficient also showed a modest increase, reaching 0,275%.

The detailed confusion matrix produced by the scorer node is reported below:

A significant improvement was observed in the model’s ability to predict the *m* category, which was not represented in the predictions of the *RProp MLP Learner*. This more balanced classification across the four categories highlights the advantage of employing the *One to Many* transformation and the Keras framework, which seems to handle class imbalance more effectively.

Although the overall accuracy of 49,159% is still below optimal expectations, this performance represents a step forward in achieving a fairer classification across all categories, especially

Table 4: Confusion Matrix for *Keras Network Learner*

	b	t	e	m
b	18.564	8.162	7.697	367
t	8.183	15.837	8.107	376
e	8.669	9.821	26.964	287
m	4.314	4.551	3.895	932

in accounting for the *m* class, which is considerably less represented in the dataset. The imbalance among the classes is a significant challenge for any classifier, and the slight improvement achieved with the *Keras Network Learner* underlines the potential of deep learning frameworks to adapt to these complexities when properly configured. This suggests that further tuning of the *epochs*, *learning rate*, and architecture of the *Keras Network Learner* could lead to even better performance and more balanced predictions.

2.3 Use of Large Language Model (LLM)

During the development of the KNIME workflows for news classification, a Large Language Model (LLM) was employed to support the understanding and improvement of the machine learning pipelines. Specifically, *ChatGPT*, based on OpenAI’s *GPT-4.5* architecture, was consulted to provide technical explanations, guidance on workflow configuration, and help in interpreting results.

The interaction began with the presentation of the initial pipeline based on the Rprop MLP Learner node, which was yielding unsatisfactory accuracy results. To address this issue, the LLM was prompted with detailed information about the dataset characteristics, class distributions, and the performance metrics obtained. An example of the prompt used was:

“Assistance is needed to improve the accuracy of a multi-layer perceptron implemented in KNIME, especially given the imbalanced classes in the dataset. What steps and considerations can help increase performance, and how can the model better handle the minority classes?”

Following this, the LLM suggested exploring a more powerful alternative: the Keras Network Learner, which supports deeper neural networks with greater flexibility. Since configuring Keras and TensorFlow within KNIME was initially unclear, further prompts guided the process. The LLM provided step-by-step instructions covering:

- Installation of Anaconda and creation of a compatible Python environment (Python 3.7 or 3.8 with TensorFlow and Keras).
- Configuration of Python preferences in KNIME to ensure the Keras nodes recognize the correct interpreter.
- Construction of the neural network using Keras input and multiple Dense layers.
- Data preparation steps, including normalization and train-test partitioning using dedicated KNIME nodes.
- Integration of the Keras Network Learner and Executor into the workflow for training and prediction.
- Post-processing of predictions with conversion to class labels and evaluation via the Scorer node.

A key insight provided by the LLM was the necessity for a “one-hot” encoding of the target classes (i.e., one column per class) when using the Keras Network Learner for multiclass classification, which contrasts with the numeric encoding used by the Rprop MLP Learner. This approach led to improved accuracy and a more balanced prediction across all classes.

The use of the LLM facilitated the iterative refinement of the workflow by clarifying concepts such as epochs, learning rate, and network architecture, and by advising on best practices for handling imbalanced data. Thus, the collaboration with the LLM proved essential in transitioning from a baseline model with limited performance to a more effective deep learning approach.

A summary of the interaction with the LLM is reported in Table 5.

Table 5: Summary of LLM Assistance in Workflow Development

Aspect	Details
Model	ChatGPT (GPT-4.5 based)
Initial Challenge	Low accuracy with Rprop MLP Learner on imbalanced data
Primary Inquiry	Methods to improve MLP accuracy and handle class imbalance
Suggested Solution	Adoption of Keras Network Learner with deeper architecture
Support Provided	Installation, environment setup, KNIME configuration, network design
Key Technical Advice	Use of one-hot encoding for multi-class output in Keras
Outcome	Increased accuracy and more balanced class predictions

3 Decision Tree

3.1 Classification Results

In the classification task of predicting the *CATEGORY* variable of news articles based solely on the *PUBLISHER* feature, a decision tree was implemented as part of the workflow.

The workflow was structured as follows: after reading and renaming the columns of the dataset, the categorical *PUBLISHER* field was converted into numeric values, since each unique publisher needed to be represented by a unique numeric identifier. Subsequently, data pre-processing steps addressed missing values and filtered the relevant columns. The dataset was partitioned into training and testing subsets, with the training subset used to train a decision tree learner whose hyperparameter *Minimum Number of Records per Node* was optimized using a parameter optimization loop.

The *Minimum Number of Records per Node* hyperparameter is critical in controlling the growth of the decision tree. It defines the minimum number of samples required to split an internal node. Setting this parameter to lower values allows the tree to become deeper and potentially overfit the data, while higher values can result in a shallower tree with a risk of underfitting. In the parameter optimization loop, this hyperparameter was varied from 1 to 10 with a step size of 1 to systematically explore its impact on accuracy.

The best performance was obtained with a value of 6 for this parameter, yielding an accuracy of approximately 63,6%.

Once the model was trained, its predictive performance was evaluated using the *Scorer* node. The confusion matrix and associated metrics showed that the decision tree achieved more balanced predictions across the four classes compared to the earlier *MLP* model used in the same task. Notably, in the MLP model, the health (m) category was not predicted at all, likely due to the strong class imbalance in the dataset where the number of samples for class *m* (45.639) was significantly smaller than those for the other classes (for instance, class *b* had 152.469 samples).

By contrast, the decision tree model demonstrated the ability to produce predictions for all categories, as detailed in Table 6.

Table 6: Confusion Matrix of the Decision Tree Learner

Predicted / True	b	t	e	m
b	23.514	3.632	6.520	1.124
t	7.474	16.678	7.201	1.150
e	5.273	2.476	36.976	1.016
m	3.893	1.970	4.389	3.440

The overall accuracy of the decision tree model was 63,608%, with 80.608 correctly classified instances and 46.118 misclassified instances. Additional metrics, such as Cohen’s Kappa coefficient (0,48), further indicated that the model performed significantly better in terms of class distribution compared to the previous MLP approach.

The improved distribution of predictions across all categories can be attributed to the fact that decision trees naturally consider splits based on categorical information, which in this case was the numeric encoding of the publisher names. Unlike the MLP, which might struggle to learn effectively from the highly imbalanced classes and limited input features, the decision tree is better equipped to find decision boundaries that include minority classes. Consequently, even with the inherent dataset imbalance, the health category (m) was finally recognized and classified by the decision tree, confirming its greater robustness in this particular classification setting.

3.2 Use of Large Language Model (LLM)

During the construction of the workflow a Large Language Model (LLM) was consulted to provide technical advice specifically related to the hyperparameter tuning process of the Decision Tree Learner. The chosen LLM, *ChatGPT*, powered by OpenAI’s *GPT-4.5* architecture, was prompted to offer guidance on selecting suitable ranges and step sizes for the *Minimum Number of Records per Node* parameter, a critical setting within the Decision Tree algorithm.

In this context, the LLM was not involved in establishing the node connections of the workflow, which were manually configured based on familiarity with KNIME’s graphical interface and data processing nodes. The main contribution of the LLM was thus to suggest an informed and empirically justified parameter search range, ensuring that the Decision Tree model would be optimized without risking overfitting or underfitting. An example of the prompt used was:

“I have a dataset with the following class distribution:

- 152.746 articles in the business category*
- 108.465 articles in the science and technology category*
- 115.920 articles in the entertainment category*
- 45.615 articles in the health category.*

I’m using a Decision Tree Learner model and want to find the optimal value for the ‘Minimum Number of Records per Node’ parameter to improve classification performance, considering the dataset imbalance. Can you help me with this?”

The following parameter configuration was adopted:

Table 7: Parameter Optimization Range for Decision Tree Learner

Parameter	Start Value	Stop Value	Step Size
Min. Records per Node	1	10	1

The parameter *Minimum Records per Node* determines the minimum number of samples that must be present in a leaf node to allow a further split in the Decision Tree. By carefully defining the range from 1 to 10 with a step size of 1, the LLM recommended a balanced search space that could reveal the optimal trade-off between model complexity and generalization. Smaller values for this parameter typically result in deeper trees that might overfit the data, while larger values tend to produce shallower trees that may underfit. The guidance from the LLM suggested that a moderate range would be sufficient to find the sweet spot.

In conclusion, the LLM’s involvement was instrumental in refining the hyperparameter tuning process, ultimately ensuring that the Decision Tree Learner’s configuration was neither too restrictive nor too permissive. This informed setup allowed the workflow to achieve a more accurate and fair classification outcome, as reflected by the improved accuracy and better-distributed confusion matrix.

4 Conclusion

This laboratory investigation explored the application of different machine learning approaches for classifying news articles, the experimental workflow compared three distinct classification approaches: the Rprop Multi-Layer Perceptron Learner, the Keras Network Learner, and a Decision Tree classifier. Each method demonstrated varying degrees of success in handling the inherent complexities of the dataset, particularly the substantial imbalance between categories. The distribution of articles across the four categories created a challenging classification scenario, with health articles representing only 10,79% of the total dataset compared to business articles comprising 36,13% of the collection.

Table 8: Performance comparison of classification models and dataset distribution

Category	Articles (%)	Rprop MLP	Keras Network	Decision Tree
Business (b)	36,13	Predicted	Predicted	Predicted
Science & Technology (t)	25,66	Predicted	Predicted	Predicted
Entertainment (e)	27,42	Predicted	Predicted	Predicted
Health (m)	10,79	Not predicted	Predicted	Predicted
Overall Accuracy		47,050%	49,159%	63,608%
Cohen’s Kappa		0,229	0,275	0,480

The Rprop MLP Learner, despite extensive parameter optimization exploring configurations between 1–3 hidden layers and 48–56 neurons per layer, achieved a maximum accuracy of 47,05% with a Cohen’s kappa of 0,229. The most significant limitation observed was the complete inability to predict the health category, resulting in zero instances assigned to this underrepresented class. This behavior highlights the sensitivity of traditional neural networks to class imbalance when trained on limited feature sets.

The Keras Network Learner demonstrated superior performance, achieving 49,159% accuracy and improving the Cohen’s kappa to 0,275. The implementation of one-hot encoding for the categorical output and the flexibility provided by the Keras framework enabled more balanced predictions across all four categories. Notably, this approach successfully predicted instances in the health category, representing a substantial improvement in handling the minority class despite the inherent dataset limitations.

The Decision Tree classifier produced the highest overall accuracy of 63,6% and a Cohen’s kappa of 0,480, outperforming both neural network-based approaches. The hierarchical, rule-based nature of decision trees proved particularly effective in managing the imbalance of class distributions, as it inherently partitions the feature space to accommodate minority classes better than purely gradient-based learners. This ability translated into more consistent predictions across all four categories, including the health class that remained elusive for the Rprop MLP model. However, while the Decision Tree model demonstrated improved performance, it still fell short of providing robust category discrimination, suggesting that further enhancements—such as ensemble approaches or feature enrichment—are required to fully capture the nuanced patterns within the dataset.

The modest performance achieved across all classification approaches underscores the fundamental limitation of using publisher information as the sole predictive feature. The publisher-based classification constraint, while realistic for certain applications, appears insufficient for achieving robust category discrimination, particularly for distinguishing health-related content from other news categories. The results suggest that incorporating additional features such as article titles, content summaries, or metadata could significantly enhance classification performance.

Furthermore, the class imbalance issue requires more sophisticated handling beyond basic stratified sampling. Advanced techniques such as synthetic oversampling, cost-sensitive learning, or ensemble methods specifically designed for imbalanced datasets could potentially improve the classification of underrepresented categories while maintaining overall model performance.

This laboratory investigation successfully demonstrated the practical implementation of machine learning workflows within KNIME while highlighting the challenges inherent in real-world classification problems. The experience emphasizes the importance of careful feature selection, appropriate handling of class imbalance, and the iterative nature of model development and optimization in achieving meaningful classification results.